

Этап 3: Создание схем базы данных

Содержание

1. Этап 3: Создание схем базы данных	2
1. 1. Установка утилиты для создания схем и миграций <code>sqlx-cli</code>	2
1. 2. Миграция.	2
1. 3. Создание схем базы данных.	2
1. 4. Создание базы данных и применение миграций.	5
Итоги	6

1. Этап 3: Создание схем базы данных

В этапе 2 вы закончили описание первоначальных моделей предметной области. На основе этих моделей, теперь можно создать схемы базы данных.

1. 1. Установка утилиты для создания схем и миграций `sqlx-cli`.

Утилита `sqlx-cli` позволяет создавать схемы базы данных и применять миграции к ним. Утилита способна работать с различными реляционными базами данных SQL: `SQLite`, `PostgreSQL`, `MySQL`.

```
cargo install sqlx-cli
```

1. 2. Миграция.

Миграция - процесс изменения структуры базы данных. Миграции позволяют обновлять схему базы данных в соответствии с изменениями в модели предметной области.

Схема базы данных - это описание структуры базы данных, включая таблицы, столбцы, индексы и ограничения.

Во время миграции:

- Выполняются скрипты SQL, которые описывают изменения в схеме базы данных.
- После выполнения миграции, схема базы данных обновляется.

1. 3. Создание схем базы данных.

Для начала, создайте в структуре вашего локального репозитория папку `migrations`, в которой будут храниться скрипты миграции.

Команда:

```
sqlx migrate add --source <папка_для_миграций> <название_миграции>
```

Создает файл миграции (`.sql`) в указанной папке с указанным названием.

Например, я хочу для следующей структуры:

```
pub struct Item {
    pub(crate) id: Uuid,
    pub(crate) name: String,
    pub(crate) price: f32,
    pub(crate) created_at: UtcDateTime,
    pub(crate) updated_at: Option<UtcDateTime>,
    pub(crate) deleted_at: Option<UtcDateTime>,
}
```

Определить таблицу, в которой будут храниться записи этой сущности.

Для этого я выполняю команду:

```
sqlx migrate add --source ./migrations items_initial
```

У меня сформируется файл миграции внутри этой папки, который будет называться как-то так:

```
20260921061659_items_initial.sql
```

Здесь в самое начало файла, можно увидеть дату и время создания миграции.

Дата и время играют здесь важную роль, потому что они позволяют управлять порядком выполнения миграций. Сначала выполняются самые старые миграции, а затем новые. Это позволяет избежать конфликтов и ошибок при выполнении миграций.

Схемы базы данных описываются просто и декларативно. Создание таблицы SQL это простой вызов:

```
CREATE TABLE IF NOT EXISTS <название таблицы>
```

Например, для структуры `Items` таблица будет выглядеть как-то так:

```
CREATE TABLE IF NOT EXISTS items (  
  id          BLOB PRIMARY KEY NOT NULL,  
  name        TEXT NOT NULL,  
  price       REAL NOT NULL,  
  created_at  TEXT NOT NULL,  
  updated_at  TEXT,  
  deleted_at  TEXT  
);
```

`BLOB` - это тип данных, который используется для хранения бинарных данных. В данном случае, `id` - это поле, которое хранит уникальный идентификатор записи. Для него можно использовать `BLOB`.

`TEXT` - это тип данных, который используется для хранения текстовых данных. В данном случае, `name` - это поле, которое хранит название товара. Для него можно использовать `TEXT`.

`REAL` - это тип данных, который используется для хранения числовых данных. В данном случае, `price` - это поле, которое хранит цену товара. Для него можно использовать `REAL`.

`created_at`, `updated_at` и `deleted_at` - это поля, которые хранят дату и время создания, обновления и удаления записи. Для них можно использовать `TEXT` или `INT`.

1. 3. 1. Создавайте для каждой таблицы отдельную миграцию.

Рекомендуется для каждого изменения схемы базы данных создавать отдельную миграцию. Например, я хочу создать новую таблицу, для структуры `Customer`. Я выполняю:

```
sqlx migrate add --source ./migrations customer_initial
```

```
CREATE TABLE IF NOT EXISTS customers(  
  id          BLOB NOT NULL PRIMARY KEY,  
  name        TEXT NOT NULL,  
  phone_number TEXT NOT NULL,  
  created_at  TEXT NOT NULL,  
  deleted_at  TEXT,  
  updated_at  TEXT  
);  
  
CREATE UNIQUE INDEX IF NOT EXISTS ux_phone_number ON customers(phone_number);
```

1. 3. 2. Индексы

В этом же файле можно объявлять и индексы. Индексы в дальнейшем используются для ускорения поиска данных в таблице. Но так же их можно применять и для создания ограничений.

Например, мне нужно, чтобы `Items` в моей системе имели только уникальные названия. Для этого я могу определить индекс уникальности значения по столбцу таблицы. Делается это вот так:

```
CREATE UNIQUE INDEX IF NOT EXISTS ux_items_name ON Items(name);
```

В зависимости от вашей предметной области, вы можете применять индексы и ограничения для создания более эффективной и строгой базы данных.

1. 3. 3. Связи

В структуре `order` находится поле `customer_id`, которое является внешним ключом к таблице `customers`.

```
pub struct Order {
    pub(crate) order_id: Uuid,
    pub(crate) item_id: Uuid, // внешняя ссылка на товар.
    pub(crate) customer_id: Uuid, // внешняя ссылка на заказчика.
    pub(crate) quantity: i32,
    pub(crate) created_at: UtcDateTime,
    pub(crate) finished_at: Option<UtcDateTime>,
}
```

Чтобы связать `Order` с `Customer`, я могу использовать внешний ключ:

```
CREATE TABLE IF NOT EXISTS orders (
    order_id          BLOB NOT NULL PRIMARY KEY,
    customer_id       BLOB NOT NULL,
    item_id           BLOB NOT NULL,
    quantity          INTEGER NOT NULL,
    created_at        TEXT NOT NULL,
    finished_at       TEXT,

    -- внешний ключ к таблице `customers`
    CONSTRAINT customers_fk
    FOREIGN KEY (customer_id) REFERENCES customers(id),

    -- внешний ключ к таблице `items`
    CONSTRAINT items_fk
    FOREIGN KEY (item_id) REFERENCES items(id)
);
```

Вы просмотрели примеры создания новых пустых миграция, описания таблиц в них и индексов по необходимости. Создайте миграции для каждой вашей сущности. У вас должно быть столько файлов миграций, сколько файлов сущностей в вашем проекте.

Например:

```

products_management/
├── migrations/
│   ├── 20260921061659_items_initial.sql
│   ├── 20260921064724_customer_initial.sql
│   ├── 20260921064732_order_initial.sql
│   └── 20260921064738_contract_initial.sql
├── src/
│   ├── entities/
│   │   ├── contract.rs
│   │   ├── customer.rs
│   │   ├── item.rs
│   │   ├── mod.rs
│   │   └── order.rs
│   └── main.rs
├── .gitignore
├── Cargo.lock
└── Cargo.toml

```

1. 4. Создание базы данных и применение миграций.

Для вашего проекта вы можете использовать следующие базы данных: `MySQL`, `PostgreSQL`, `SQLite`.

`MySQL` и `PostgreSQL` - это реляционные базы данных, являющиеся отдельным сервисом. Ваш разрабатываемый проект будет с ним общаться по сети. Поэтому, если вы хотите использовать одну из этих баз данных, вам нужно развернуть соответствующий локальный сервер на вашем компьютере. Так же, вы можете развернуть базу данных внутри Docker-контейнера.

Простой путь - это использовать `SQLite` для создания учебного проекта. Это удобно, так как `SQLite` - это локальная база данных, которая не требует отдельного сервера. Физически, она является файлом на вашем компьютере.

Далее будут описаны этапы создания базы данных `SQLite` и применения миграции.

1. 4. 1. Создание базы данных `SQLite`

1. Создайте папку на уровне `src` и назовите её `databases`. Там будут `.db` файлы с базами данных.
2. Создайте файл `.env` в корне проекта и добавьте в него следующие строки:

```
DATABASE_URL=sqlite://databases/<название_файла.db>
```

Например:

```
DATABASE_URL=sqlite://databases/data.db
```

3. Выполните команду:

```
sqlx db create
```

У вас должен сформироваться файл `.db` в папке `databases`.

4. Затем, нужно применить сами миграции (скрипты `.sql`, которые мы писали ранее), чтобы создать структуру таблиц в базе данных.

```
sqlx migrate run --source <путь к папке с миграциями>
```

1. 4. 2. Если не получается и происходит ошибка, связанная с `DATABASE_URL`, можно попробовать использовать аргумент `--database-url` или `-D` для ваших команд:

```
sqlx db create -D sqlite://databases/data.db
```

```
sqlx migrate run --source <путь к папке с миграциями> -D sqlite://databases/data.db
```

После выполнения всех шагов, у вас должна быть следующая похожая структура внутри директории:

```
products_management/
├── database/
│   └── data.db
├── migrations/
│   ├── 20260921061659_items_initial.sql
│   ├── 20260921064724_customer_initial.sql
│   ├── 20260921064732_order_initial.sql
│   └── 20260921064738_contract_initial.sql
├── src/
│   ├── entities/
│   │   ├── contract.rs
│   │   ├── customer.rs
│   │   ├── item.rs
│   │   ├── mod.rs
│   │   └── order.rs
│   └── main.rs
├── .gitignore
├── Cargo.lock
└── Cargo.toml
```

Зафиксируйте ваши изменения в системе контроля версий (например, Git). И отправьте их на удаленный репозиторий.

```
git add .
git commit -m "added initial entities database schemas"
git push origin main
```

Итоги

Вы установили утилиту `sqlx-cli` для работы с базой миграциями баз данных SQL.

Вы создали файлы миграций, для создания таблиц под ваши сущности.

Вы применили миграции и сформировали `.db` файл для базы данных.